

A Semi-Supervised learning Approach to Quality-Based Web Service Classification

A thesis submitted in partial fulfillment of the requirements for
the award of the degree of

B.Tech

In

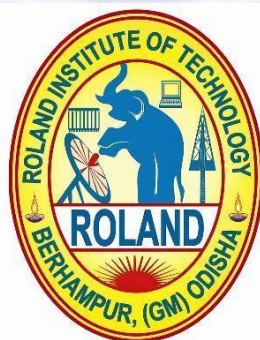
Computer Science And Engineering

By

Sourava Pradhan (Regd No. 2201204095)

Guided By

Prof. Manik Chand Patnaik



COMPUTER SCIENCE AND ENGINEERING

ROLAND INSTITUTE OF TECHNOLOGY GOLONTHARA OD 761008

May 2026

In memory of

Late Dr J Surya Rao “Founder Chairman” RIT Our deepest gratitude.

BONAFIDE CERTIFICATE

This is to certify that the project titled A Semi-Supervised Learning Approach To Quality Based Web Service Classification is a bonafide record of the work done by

Sourava Pradhan (Regd No 2201204095)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** of the **ROLAND INSTITUTE OF TECHNOLOGY, GOLONTHARA, ODISHA 761008**, during the year 2026-27

Prof. Manik Chand Patnaik
Guide

Prof. Sunil Kumar Sahu
Head Of The Department

Dr Gunamani Jena
Principal

Project Viva-voce held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENTS

We would like to express our deepest gratitude to the following people for guiding us through this course and without whom this project and the results achieved from it would not have reached completion.

Prof. Manik Chand Patnaik, Professor, Department of Computer Science and Engineering, for helping us and guiding us in the course of this project. Without his guidance, we would not have been able to successfully complete this project. His patience and genial attitude is and always will be a source of inspiration to us.

Prof. Sunil Kumar Sahu, the Head of the Department, Department of Computer Science and Engineering, for allowing us to avail the facilities at the department.

We are thankful to the Principal **Dr Gunamani Jena** for providing the facility in this institution.

We are also thankful to the Honorable Chairperson Mrs J Jayalakshmi for motivating, arranging training and internship related to the Project.

We are also thankful to the faculty and staff members of the Department of Computer Science and Engineering, our individual parents and our friends for their constant support and help.

INDEX

S.NO.	CHAPTER PAGE	PAGENO
1	INTRODUCTION	2
2	HOW MACHINE LEARNING WORKS	3
2.1	MACHINE LEARNING ALGORITHMS	3
2.2	TYPES OF MACHINE LEARNING METHODS	4
2.3	APPLICATIONS OF MACHINE LEARNING	5
2.4	WHAT IS DEEP LEARNING?	6
2.5	HOW DEEP LEARNING WORKS	7
3	LITERATURE SURVEY	8
4	SYSTEM ANALYSIS	9
4.1	EXISTING SYSTEM	9
4.2	PROPOSED SYSTEM	9
5	FUNCTIONAL REQUIREMENTS	11
5.1	SYSTEM FUNCTIONAL REQUIREMENTS	11
6	NON-FUNCTIONAL REQUIREMENTS	13
6.1	REQUIREMENTS	13
7	FEASIBILITY STUDY	15
8	REQUIREMENT SPECIFICATION	16
8.1	HARDWARE REQUIREMENTS	16
8.2	SOFTWARE REQUIREMENTS	16
9	SYSTEM DESIGN	17
9.1	METHODOLOGY	17
9.2	DATASET	20

9.3	SYSTEM ARCHITECTURE	21
10	UML DIAGRAMS	22
11	TECHNOLOGY DESCRIPTIONS	28
11.1	PYTHON FEATURES	29
11.2	PYTHON APPLICATIONS	30
11.3	WHY PYTHON	31
12	SAMPLE CODE	32
13	TESTING	37
13.1	INTRODUCTION	37
13.2	SYSTEM TESTING AND IMPLEMENTATION	37
13.3	TESTING TECHNIQUES	38
13.4	TESTING STRATEGIES	38
14	SCREENSHOTS	40
15	CONCLUSION	45
16	GLOSSARY	46
17	BIBLIOGRAPHY	48

LIST OF FIGURES

Figure 1 Architecture.....	21
Figure 2 Use case Diagram	23
Figure 3 Class Diagram.....	24
Figure 4 Sequence Diagram	25
Figure 5 Collaboration Diagram.....	26
Figure 6 Activity Diagram	27

LIST OF SCREENS

Screen 1 Dataset Class Label Graph	40
Screen 2 Decision Tree.....	40
Screen 3 SVM	41
Screen 4 Logistic Regression	41
Screen 5 Random Forest	42
Screen 6 SSL-WSC	42
Screen 7 CatBoost	43
Screen 8 Home Page.....	43
Screen 9 Admin Login	44
Screen 10 Predicted Web service as Silver	44

ABSTRACT

The Internet has evolved into a vast and dynamic platform for sharing services, where web service brokers play a critical role in helping users select the most appropriate service among numerous similar alternatives. With the rapid growth of web services, evaluating service quality has become increasingly important to ensure optimal user satisfaction and system performance. However, identifying and collecting quality-based data is often challenging, time-consuming, and requires significant manual effort. This limitation creates a need for intelligent and automated approaches to service classification and recommendation.

Machine learning techniques provide an effective solution by enabling efficient classification, prediction, and recommendation of high-quality services. In this context, this study introduces the SSL-WSC (Semi-Supervised Learning for Web Service Classification) algorithm, which leverages semi-supervised self-training to address the scarcity of labeled data. Unlike traditional supervised learning methods that rely heavily on large labeled datasets, the proposed approach effectively utilizes both labeled and unlabeled data to improve classification performance.

The SSL-WSC algorithm follows a two-step process consisting of scoring and dynamic thresholding. In the scoring phase, services are evaluated based on multiple quality attributes, while in the dynamic thresholding phase, the model iteratively selects high-confidence predictions from unlabeled data to expand the training set. This self-training mechanism enhances the model's learning capability and reduces dependency on manually labeled data.

The performance of the proposed method was evaluated using quality features from the QWS (Quality of Web Services) dataset. Experimental results demonstrate that the semi-supervised approach significantly outperforms traditional supervised methods in terms of accuracy, efficiency, and scalability. Furthermore, the model shows improved generalization and robustness when dealing with incomplete or partially labeled datasets.

Overall, this research highlights the effectiveness of semi-supervised learning in web service classification and provides a scalable and efficient solution for service recommendation in real-world environments.

1. INTRODUCTION

The internet provides a platform for sharing services, allowing users to access various resources and tools through web services. Web services are software applications designed to enable machine-to-machine communication over a network. These services follow a **Service-Oriented Architecture (SOA)** and use protocols like **SOAP (Simple Object Access Protocol)** to exchange information. However, as the number of available web services increases, selecting the best service becomes challenging due to variations in service quality.

Background and Motivation

Web services play a crucial role in modern technology, enabling businesses to offer their products and services online. They are widely used in industries such as e-commerce, social media, video streaming, and financial services. The main challenge faced by users is identifying the best service among multiple similar services. While services may offer similar functions, their quality can differ significantly. Service quality is measured using non-functional attributes such as:

- **Response time** – The time taken by the service to respond to a request.
- **Availability** – The percentage of time a service is accessible.
- **Throughput** – The number of requests a service can handle per second.
- **Security** – Protection against unauthorized access and data breaches.
- **Success rate** – The percentage of successful service executions.

Challenges in Web Service Classification

1. **Incomplete and Unlabeled Data** – Most datasets contain a large amount of unlabeled data, making it difficult for supervised learning models to function effectively.
2. **Similarity of Services** – Many web services offer similar functions, making it hard to distinguish them based on functional attributes alone.
3. **Dynamic Nature of Web Services** – The performance and availability of services can change over time, requiring continuous updates to classification models.
4. **Cost of Manual Labeling** – Labeling large datasets manually requires significant human effort and increases operational costs.
5. **Need for High Accuracy** – Classification errors can lead to poor service recommendations, impacting user satisfaction and business outcomes.

To overcome these challenges, a hybrid approach combining supervised and unsupervised learning is required. This led to the development of **Semi-Supervised Learning (SSL)** techniques, which use a small set of labeled data along with a larger set of unlabeled data to train models.

2. HOW MACHINE LEARNING WORKS

To begin with, this and computer science and, more particularly, AI and its subset called machine learning focuses on the collection of data and making use of algorithms to accomplish the objective of replicating the human learning processes more accurately rather than attempting to entirely imitate them. This is critical for data science as it allows systems to be able to uncover complex relationships within data that can make predictions or classifications automatically, and later using these relationships to make decisions within a business or any other establishment. Such insights are essential, especially for data mining initiatives and can have critical impact on major business development indices.

However, as the volume of big data is increasing so does the need for data scientists who understand the business perspective and the problems that needs solving and how data can provide the answers to the problems. This is where machine learning becomes relevant as it provides an opportunity to solve all these problems by automating the process through the use of statistical methods in training algorithms to recognize underlying relationships or patterns with data.

2.1 Machine Learning Algorithms

There are three important principles on which machine learning algorithms evolve.

1. A Decision Process.

This is the formulization of the problem which has to be solved by the machine learning. The problem solving begins with the identification of what the problem is and this means stating what are the objectives of the problem.

2. An Error Function

The algorithm's predictions are ascertained with the aid of the error function. The error function is an evaluation tool, which verifies the efficiency of the model based on the output in relation to the available known situations and indicates places where improvements can be made in the model.

3. A Model Optimization Process

During the optimization process, the already defined model is employed to enhance its accuracy by modifying parameters, for instance, weights, that determine the alignment of the model with the data. The model conducts its own evaluation, causes some changes which are afterward subjected to further evaluation, and repeats everything until it fully satisfies itself with the accuracy of its prediction, or the process exceeds some limits preset.

2.2 Types of Machine Learning Methods

Supervised Machine Learning

Initiating with labelled data leads to supervised learning algorithms, thereby informing the input and the output. In this manner, the model is able to understand the rules and the relationships of the data so as to classify patterns and provide predictions of unseen data. Adding the data would energise the model to make weight adjustments to the parameters so as to better the performance during the training phase. Cross-validation is adopted for balancing cases where overfitting (not too general) and underfitting situations (too general) are avoided.

Unsupervised Machine Learning

On the contrary, unsupervised learning involves working with unlabelled sets of data. Such algorithms examine the input data to discover and cluster the data into groups or structures that were not taught to them. This makes it very useful for exploratory data analysis, for market segmentation, cross-marketing, or for recognizing images and patterns.

Semi-Supervised Learning

To do this, semi-supervised learning bridges supervised and unsupervised learning by employing a few labeled data and lots of unlabeled data. The final understanding of the academic efforts is achieved through feature extraction and image collection from the labeled files, hence the model is able to understand the unpreferred files. This model proves useful in situations where the labeled data is scarce or the processes of labeling data is expensive or infeasible.

2.3 Applications of Machine Learning

1. Voice Identification

Automatic Speech Recognition (ASR) also known as voice recognition is that aspect of natural language processing (NLP) wherein spoken words are converted to written texts. This technology is quite widely used on portable devices, for instance, mobile phones, where people are able to do voice searches on devices such as Siri and can also send text messages without using their hands.

2. Customer support

Customer interaction and engagement are being revolutionized by AI powered chatbots. Customers can be assisted on sites and across social media platforms. These AI powered bots can answer frequently asked questions about shipping conditions, give fitting and size suggestions, and recommend goods to buy, all of which help to maximize customer interaction. These can include ecommerce virtual agents, slack and facebook messenger bots, and voice interface systems.

3. Vision AI

With the help of computer vision, systems can analyze faces, videos and other visual aspects to gain an understanding of the situation at hand. Unlike simple image recognition, such approaches can view pictures and videos and consider them for decision making. These include:

- Tagging of pictures posted on social media sites.
- Healthcare radiology imaging.
- Automobiles with automated driving features in the automobile sector.

4. Virtual shopping assistants

Past behavioral data of users is outputted by an AI algorithm and then analyzed to find valuable insights which can be used to strategically target users. This is most common in e-commerce stores where clients are offered recommendations about items that can be purchased in addition to what the user initially intended to buy.

5. Stock trading robots

The new trading platforms that have been developed focus on algorithmic trading of stocks while an AI powered engine assembles the stock portfolio. Daily, these computers can place hundreds of thousands or millions of trades, making them suitable for stock markets.

2.4 WHAT IS DEEP LEARNING?

Deep learning is a subfield of AI that engages people's attention the most today. This is the case because deep learning allows the automation of tasks such as image classification, speech recognition, object detection, and content description. In short, deep learning allows machines to comprehend data in a more advanced way.

For example, deep learning is used in image classification, speech recognition, object detection, and content analysis. The exponential growth can be attributed to several factors:

Key Drivers of Deep Learning Advancements

1. Algorithmic Improvements

The innovations in the algorithms have drastically increased the usability of the deep learning models.

2. New Machine Learning Approaches

The methodologies have evolved over the years, and that has retained the prediction accuracy of the models as well as the decisions made.

3. Specialized Neural Networks

New formats of the neural networks have been created to enable the performance of certain functions such as document translation or photos categorization to these networks.

4. Abundance of Metadata

The availability of such data is practically a revolution and includes the following types:

- Streaming data from IoT devices;
- Textual data from social media;
- Professional notes like physicians' notes and investigative transcriptions.

These data sources constitute a solid basis for constructing and deep-layered neural networks.

2.5 How Deep Learning Works

The deep learning paradigm has the ability to bring revolution in advancement of the field analytics. Deep learning treats the problem differently, as in this approach instead of being explicitly coded the whole program detailing how the problem can be solved, the clear problem, in this case, is to teach the computer to learn how to solve it by itself.

Analytical Methodology In Deep Learning

In traditional analytics, the steps involved in the approach tend to include the following:

1. **Feature Engineering:** Taking out an variable (features) or picking out variables from the data set and recreating ones that can adequately describe the issue.
2. **Model Development:** All the phases of the study until a particular model is completed and picked on.
3. **Parameter Fitting:** Creating the best model that represents the data by changing the parameters of the model.

Although this approach can yield satisfactory predictive systems, it has weaknesses. The performance of the system is highly influenced by the quality of the features and the model. For example, while constructing a model for detecting fraud, one would say:

- You start with parameters that are already set and apply data manipulations in order to generate a model.
- It follows that this process may produce several thousands of variables, which will involve painstaking work to sift through this mass and select the significant ones.
- Inserting new information requires the application of the entire feature engineering exercise all over again.

3. LITERATURE SURVEY

1. Graph Convolutional Networks for Web Service Classification (Li et al., 2021)

Li et al. proposed the use of Graph Convolutional Networks (GCNs) to model relationships between web services. The method effectively captured complex dependencies, leading to improved classification accuracy. However, the implementation required significant domain expertise and was complex to deploy in real-world applications.

2. Supervised Machine Learning Models for Web Service Classification (Chipa et al., 2019)

Chipa et al. compared supervised learning models, such as K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Decision Trees, for web service classification. Their findings indicated that decision trees outperformed other models in terms of interpretability. However, all supervised models struggled with unseen services, emphasizing the need for semi-supervised approaches.

3. Reinforcement Learning for Web Service Classification (Sheng et al., 2021)

Sheng et al. applied reinforcement learning techniques to filter irrelevant data and improve classification accuracy. The study demonstrated the adaptability of reinforcement models in dynamic environments. However, the model required extensive tuning to optimize classification performance.

4. Generative Adversarial Networks (GANs) for Web Service Classification (Xie et al., 2023)

Xie et al. developed a novel classification method using GANs to learn service representations. The approach enhanced classification accuracy across heterogeneous datasets. However, GAN-based models suffered from training instability and required significant computational resources.

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

Many companies will publish web services such as Online Bill Payments, E-Commerce Payment and many more. Among all companies some will provide services with cheaper cost and best services and for users it difficult to find out which services are best at cheaper cost. To identify best services many machine learning algorithms are introduced but this algorithms required huge amount of label dataset to achieve good accuracy and its accuracy will be degrade when dataset has fewer records.

Disadvantages of Existing system

- Quality inconsistencies
- Requires extensive data for accuracy
- Difficult to scale with demand
- Security vulnerabilities
- Continuous updates and monitoring needed.
- Inconsistent ranking results
- Requires substantial processing power.

4.2 PROPOSED SYSTEM

To overcome from above issue author of this paper proposing concept called Semi-Supervised Web Service Classification (SSL-WSC) algorithm which can train on few label dataset and can predict labels for un-label instances.

Machine learning algorithms are of three types such as Supervise, unsupervised and semi-supervise.

- 1) Supervise algorithms required dataset with labels for training
- 2) Un-supervise algorithms required dataset without labels and can predict labels for dataset. Example of unsupervised algorithms are K-MEANS Clustering, CMEANS and many more

- 3) Semi-Supervise algorithms required few size of label dataset and can predict labels for large size of un-label dataset.

In propose work author utilizing Semi-supervise algorithm to predict web-services type. Each web service is classify under 4 different class labels such as PLATINUM, GOLD, SILVER and BRONZE. All best working services are under platinum and gold and worst under silver and bronze.

To assign class label accurately author of this paper employing different distance functions such as MAHALANOBIS, MANHATTAN and MINOWSKI. We are employing best distance function called MAHALANOBIS and this distances are further calculated using 4 different scores.

- 1) Score1: this score will be sum of absolute difference between all class labels
- 2) Score2: sum of distances of all class labels
- 3) Score3: minimum of all scores
- 4) $\text{Score4} = \text{score1} + \text{score2} / \text{score3}$ (confidence score)

Advantages of Proposed System

1. Efficient data classification
2. Improved accuracy
3. Adjusts dynamically to data changes
4. Enhanced service selection
5. Handles large datasets efficiently
6. Speeds up the selection process.
7. Quality-based ranking
8. Maximizes computational resource use.

5. FUNCTIONAL REQUIREMENTS

Functional requirements form an important part in the sphere of software and systems engineering as it determines the conditions which must be fulfilled in order for the system or any of its components to perform their particular tasks. Explain how inputs are transformed into outputs and what the system is expected to achieve once it is completed. Analytic computations, data creation and even some operational processes would be some instances of these requirements. Functional requirements concern the system in its aspects including but not limited to the performance capabilities while non-functional requirements address specific components and capabilities within a system, such as performance, security, or reliability.

Functional requirements are typically put forth as the condition “the system must do ,” whereas for non-functional requirements the statement is put in the form of “the system shall be .” These functional requirements are provided in the system design, while non- functional requirements are maintained in the systems’ architectures. Therefore functional requirements controls the application architecture, and non-functional requirements control the technical architecture.

For the cases above and in real life, functional requirements can be confirmed and further refined through the use cases where the situation is first described. Clients provide their expectations and analysts try to observe, interact and record these expectations. These use cases are instrumental in generating the functional requirements and ensuring that the right functions are performed by the system. These steps of analyses, modeling and validation help to ensure that the requirements are met with expectation of the stakeholders and are possible to be met in reality.

5.1 SYSTEM FUNCTIONAL REQUIREMENTS

Data Collection:

The system should collect labeled and unlabeled data from the QWS dataset for training and testing.

Data Preprocessing:

The system should clean, normalize, and standardize data to prepare it for training.

Model Training:

The system should train a base classifier using a small set of labeled data.

Self-Training:

The system should predict labels for unlabeled data and add high-confidence samples to the training set.

Distance-Based Selection:

The system should calculate distances (e.g., Mahalanobis, Manhattan) to select similar samples.

Dynamic Thresholding:

The system should adjust the threshold dynamically based on classification confidence.

Model Evaluation:

The system should evaluate performance using accuracy, F1-score, and precision.

Adaptive Learning:

The system should update the model dynamically with new data.

Multi-Class Classification:

The system should classify web services into Platinum, Gold, Silver, and Bronze categories.

Performance Reporting:

The system should generate detailed reports on classification results and model accuracy.

6. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements of a system indicate the maintained quality or response time of a system or application. For instance, a user may be configured to a system and once selected the system is expected to respond within a certain time frame: 3 seconds for 10,000 users concurrent. In a brief statement, these types of requirements explain how the system is going to fulfill its operations.

Just like deadlines in real life, they are a must while working on any product or delivering services. Many would refer to these as constraints, as these do set limitations or boundaries within which the design needs to operate. One notion of philosophy in design states ‘out of sight, out of mind’ and these graphic leaders allow exactly that so the consumer focuses on the primary objectives. Without NFRs, consumers could easily become unsatisfied due to the inefficiency of a product.

6.1 REQUIREMENTS

Performance:

The system should achieve high accuracy (over **90%**) in classification under various conditions.

Scalability:

The system should handle large datasets without compromising performance.

Reliability:

The system should provide consistent classification results across different scenarios.

Security:

The system should protect data from unauthorized access through encryption and secure storage.

Usability:

The user interface should be easy to navigate and interpret for both technical and non-technical users.

Maintainability:

The system should allow for easy updates and modifications without disrupting existing functionality.

Portability:

The system should run smoothly on different platforms and operating systems.

Adaptability:

The system should dynamically adjust classification thresholds based on data patterns.

Transparency:

The system should provide clear explanations for classification decisions.

Availability:

The system should have an uptime of at least **99%** to ensure continuous access.

7. FEASIBILITY STUDY

The feasibility study succinctly addresses whether the envisaged proposal is workable by presenting an outline and cost figures for the project. This phase is very important in determining whether the system is in concurrence with the objectives of the organization and whether it is appropriate. For purposes of carrying out a feasibility study, there need to be always criteria to be satisfied, in this case, that of understanding the system requirements. It covers the following areas:

1. Economic Feasibility:

This assesses the economic worth of the system. The expenditures incurred in the engagement of such R&D activities should be able to cover the cost incurred. The system is constructed in a way that eliminates many unnecessary costs by using free alternatives and restricting tailor made procurements.

2. Technical Feasibility:

This assesses how the system's technical needs correspond to what is available. The completion of the system development avoids many technical specifications to minimize the alteration of the system at the time of implementation.

3. Social Feasibility:

This measures a system in terms of the aptness or readiness of users to use the system. Training and tutorials build users' self familiarity with the system aiding their seamless adoption of the system in their day to day work processes. User encouragement to give feedback increases the chances of enhanced adoption experience.

8. REQUIREMENT SPECIFICATION

8.1 Hardware Requirements

- PROCESSOR - Intel Core i3/i5
- RAM - 8 GB
- HARD DISK (SSD) - 512 GB

8.2 Software Requirements

- Operating system : Windows 11/ 10
- Coding Language : PYTHON, Jupyter Notebook

9. SYSTEM DESIGN

9.1 METHODOLOGY

1. Data Collection:

- The **QWS dataset** is used, containing multiple web services that are classified into four categories based on their quality. This dataset includes both labeled and unlabeled data, making it suitable for SSL.

2. Data Preprocessing:

- Data preprocessing involves handling missing values and shuffling the dataset to ensure that the data is clean and properly distributed for model training.

3.1 Decision Tree Classifier

- **Parameters:** A maximum depth of 3 and a maximum of 1 feature considered for splitting at each node.
- **Rationale:** Decision trees are simple yet powerful models that can capture non-linear relationships.

3.2 Support Vector Machine (SVM)

- **Parameters:** Default kernel (RBF), probability estimates enabled.
- **Rationale:** SVMs are effective in high-dimensional spaces and are robust against overfitting, especially in cases where the number of dimensions exceeds the number of samples.

3.3 Logistic Regression

- **Parameters:** Maximum iterations set to 1500, solver set to 'liblinear', tolerance set to 0.2.
- **Rationale:** Logistic regression is a linear model suitable for multiclass classification when the relationship between the features and the target is approximately linear.

3.4 Random Forest Classifier

- **Parameters:** Maximum depth of 1, 10 estimators (trees), and a maximum of 1 feature considered for splitting.

- **Rationale:** Random Forest is an ensemble method that reduces overfitting by averaging multiple decision trees, enhancing predictive accuracy.

3.5 Multi-Layer Perceptron (MLP) Classifier

- **Parameters:** Solver set to 'adam', alpha set to 0.001, hidden layer sizes set to (64, 4), random state set to 1.
- **Rationale:** MLPs are neural networks capable of modeling complex patterns through multiple layers of neurons.

3.6 XGBoost Classifier

- **Parameters:** Objective set to 'multi', learning rate set to 0.1, maximum depth set to 50, 15 estimators, evaluation metric set to 'mlogloss', random state set to 42, minimum child weight set to 50.

Rationale: XGBoost is a powerful ensemble algorithm that uses gradient boosting to optimize model performance.

4. Proposed Semi-Supervised Learning Approach (SSL-WSC)

A semi-supervised learning algorithm is proposed to enhance classification accuracy by leveraging both labeled and unlabeled data.

5. Performance of Traditional Classifiers

Traditional classifiers were applied to the dataset to compare their performance with the proposed SSL-WSC algorithm.

Key Results:

1. **SVM:** Achieved **24% accuracy**, showing poor performance on a small labeled dataset.
2. **Logistic Regression:** Reached **56% accuracy**, slightly better but still not optimal.
3. **Random Forest:** Achieved **54% accuracy**, illustrating the limitations of conventional classifiers when labeled data is scarce.

6. SSL-WSC Performance

The SSL-WSC algorithm was evaluated on the QWS dataset, showing significant improvements over traditional classifiers in terms of accuracy and other key metrics.

SSL-WSC Results:

1. **Accuracy:** The SSL-WSC algorithm achieved **68% accuracy**, outperforming SVM, Logistic Regression, and Random Forest.
2. **Confidence Score Assignment:** The algorithm successfully assigned labels to unlabeled data points based on their confidence scores, significantly improving classification accuracy.

7. Extension with CATBOOST

To further improve accuracy, the project extended the SSL-WSC model with **CATBOOST**, a high-performance gradient boosting algorithm known for handling categorical data effectively.

Key Results from CATBOOST:

1. **Accuracy:** CATBOOST achieved an impressive **95% accuracy**, significantly outperforming the original SSL-WSC model.

Confusion Matrix: The confusion matrix for CATBOOST shows that the model correctly predicted the majority of class labels, with minimal incorrect predictions.

8. Evaluation Metrics

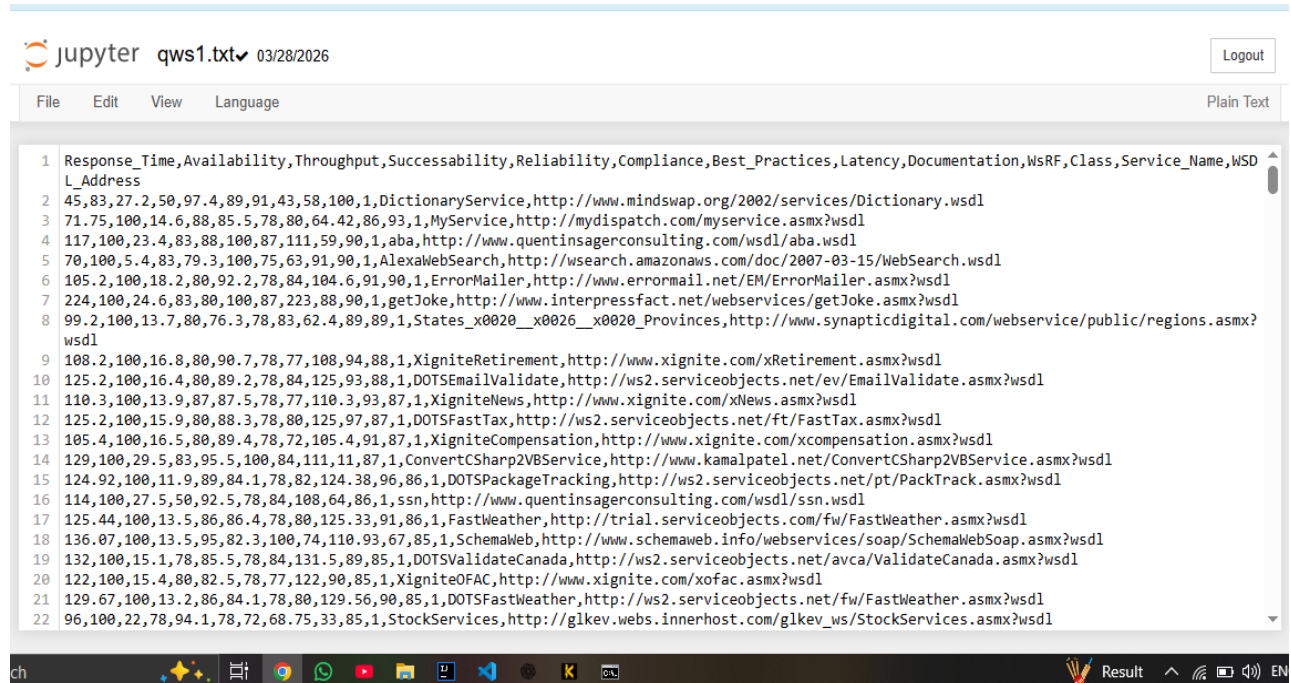
The performance of the SSL-WSC and CATBOOST models was evaluated using several key metrics to ensure that they provided accurate and reliable results.

Key Evaluation Metrics:

1. **Accuracy:** Measures the overall correctness of the model's predictions.
2. **Precision & Recall:** Precision evaluates the proportion of correct positive predictions, while recall measures the model's ability to identify all relevant instances.
3. **F1-Score:** Provides a balanced measure of precision and recall, particularly important when dealing with imbalanced data.

9.2 DATASET:

The QWS dataset is a widely used dataset for evaluating web service quality. It contains multiple attributes related to service quality, such as response time, availability, and throughput. You can find and download the QWS dataset by searching for "QWS dataset download" on Google or visiting popular research repositories like Kaggle or UCI Machine Learning Repository.



The screenshot shows a Jupyter Notebook interface with a file named 'qws1.txt' opened. The notebook contains a list of 22 web services, each with a line number and a comma-separated list of attributes. The attributes include Response_Time, Availability, Throughput, Successability, Reliability, Compliance, Best_Practices, Latency, Documentation, WsRF, Class, Service_Name, WSDL_Address, and others. The services are listed in a table-like format with line numbers 1 through 22.

```
1 Response_Time,Availability,Throughput,Successability,Reliability,Compliance,Best_Practices,Latency,Documentation,WsRF,Class,Service_Name,WSDL_Address
2 45,83,27.2,50,97.4,89,91,43,58,100,1,DictionaryService,http://www.mindswap.org/2002/services/Dictionary.wsdl
3 71.75,100,14.6,88,85.5,78,80,64.42,86,93,1,MyService,http://mydispatch.com/myservice.asmx?wsdl
4 117,100,23.4,83,88,100,87,111,59,90,1,aba,http://www.quentinsagerconsulting.com/wsdl/aba.wsdl
5 70,100,5.4,83,79.3,100,75,63,91,90,1,AlexaWebSearch,http://wsearch.amazonaws.com/doc/2007-03-15/WebSearch.wsdl
6 105.2,100,18.2,80,92.2,78,84,104.6,91,90,1,ErrorMailer,http://www.errormail.net/EM/ErrorMailer.asmx?wsdl
7 224,100,24.6,83,80,100,87,223,88,90,1,getJoke,http://www.interpressfact.net/webservices/getJoke.asmx?wsdl
8 99.2,100,13.7,80,76.3,78,83,62.4,89,89,1,States_x0020_x0020_Provinces,http://www.synapticdigital.com/webservice/public/regions.asmx?wsdl
9 108.2,100,16.8,80,90.7,78,77,108,94,88,1,XigniteRetirement,http://www.xignite.com/xRetirement.asmx?wsdl
10 125.2,100,16.4,80,89.2,78,84,125,93,88,1,DOTSEmailValidate,http://ws2.serviceobjects.net/ev/EmailValidate.asmx?wsdl
11 110.3,100,13.9,87,87.5,78,77,110.3,93,87,1,XigniteNews,http://www.xignite.com/xNews.asmx?wsdl
12 125.2,100,15.9,80,88.3,78,80,125,97,87,1,DOTSFastTax,http://ws2.serviceobjects.net/ft/FastTax.asmx?wsdl
13 105.4,100,16.5,80,89.4,78,72,105.4,91,87,1,XigniteCompensation,http://www.xignite.com/xcompensation.asmx?wsdl
14 129,100,29.5,83,95.5,100,84,111,11,87,1,ConvertCSharp2VBService,http://www.kamalpatel.net/ConvertCSharp2VBService.asmx?wsdl
15 124.92,100,11.9,89,84.1,78,82,124.38,96,86,1,DOTSPackageTracking,http://ws2.serviceobjects.net/pt/PackTrack.asmx?wsdl
16 114,100,27.5,50,92.5,78,84,108,64,86,1,ssn,http://www.quentinsagerconsulting.com/wsdl/ssn.wsdl
17 125.44,100,13.5,86,86.4,78,80,125.33,91,86,1,FastWeather,http://trial.serviceobjects.com/fw/FastWeather.asmx?wsdl
18 136.07,100,13.5,95,82.3,100,74,110.93,67,85,1,SchemaWeb,http://www.schemaweb.info/webservices/soap/SchemaWebSoap.asmx?wsdl
19 132,100,15.1,78,85.5,78,84,131.5,89,85,1,DOTSValidateCanada,http://ws2.serviceobjects.net/avca/ValidateCanada.asmx?wsdl
20 122,100,15.4,80,82.5,78,77,122,90,85,1,XigniteOFAC,http://www.xignite.com/xofac.asmx?wsdl
21 129.67,100,13.2,86,84.1,78,80,129.56,90,85,1,DOTSFastWeather,http://ws2.serviceobjects.net/fw/FastWeather.asmx?wsdl
22 96,100,22,78,94.1,78,72,68.75,33,85,1,StockServices,http://glkev.webs.innerhost.com/glkev_ws/StockServices.asmx?wsdl
```

9.3 SYSTEM ARCHITECTURE:

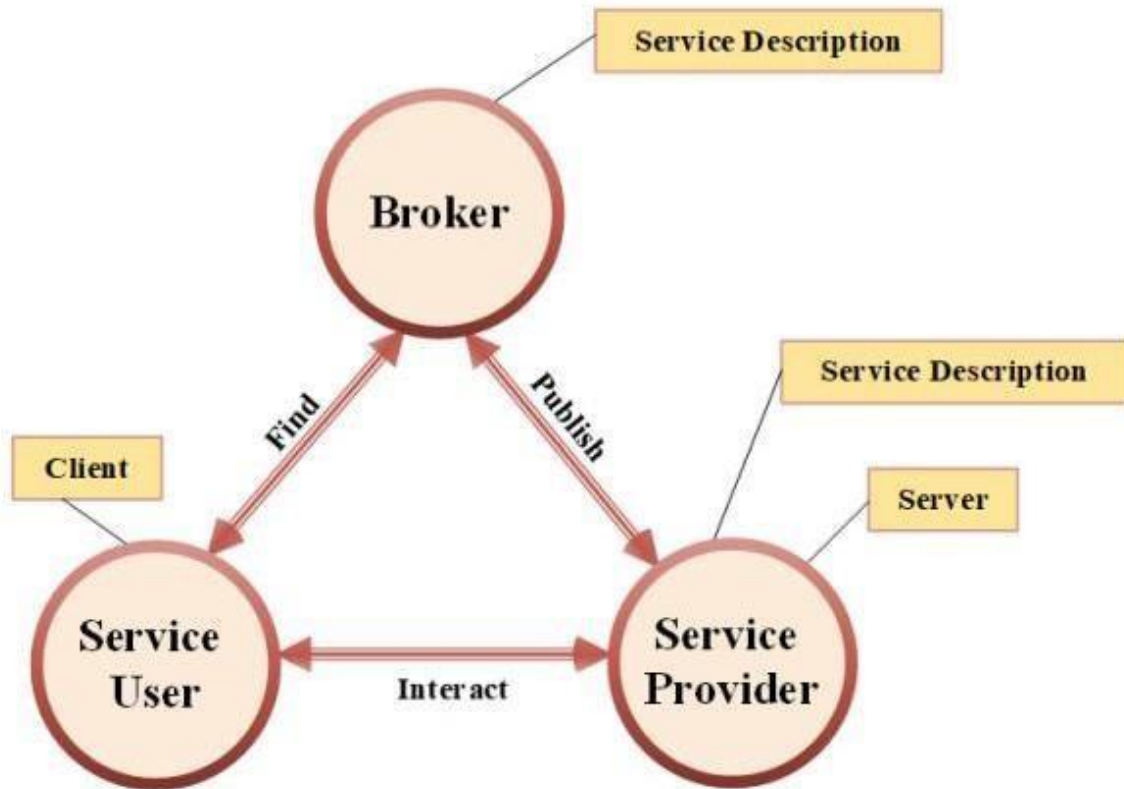


Figure 1 Architecture

10. UML DIAGRAMS

UML is a model while UML is a modeling language. Unified Modeling Language is an internationally accepted modeling palette which makes it easy to visually express plans, implemented ideas and facts in terms of a system- OO. It is a language, so it can be spoken and written. It is a standardized language and there is no national ownership.

UML comprises of two basic aspects: a Meta-model and Notation. This has further enabled the specification, visualization, construction and documentation of software artifacts. It moreover fosters business modeling and other non-software systems, placing emphasis graphical notations for better communication of designs of software systems.

UML adopts the best engineering practices which have been found useful in the modeling of complex and large systems. It is a must in object oriented software engineering for the entire process to be clear, consistent and understandable.

Goals of UML

The primary objectives of UML include:

1. Enhancing the ability to create and communicate meaningful models by providing an appropriate visual modeling language.
2. Introducing provisions for the extendibility and specialization of the basic concepts.
3. Guaranteeing freedom with respect to particular programming languages and specific development methodologies.
4. Providing a helpful underpinning to make clear the language.
5. Facilitating the development of the market of the object-oriented tools.
6. Making it possible to use more sophisticated development concepts such as collaborations, frameworks, patterns and components.

10.1 USE CASE DIAGRAM

A use case diagram in Unified Modeling Language (UML) belongs into class of behavioral diagrams and is obtained through use-case analysis. It graphically represents the functionality of the system by depicting the interactions of the actors towards the achievement of their goals which are modeled as use cases. And, it provides an overview of the dependencies between these use cases.

The main objective of use case diagrams is to indicate what tasks the system performs for each of its actors, so that, the tasks and responsibilities of the actors in the system are better understood.

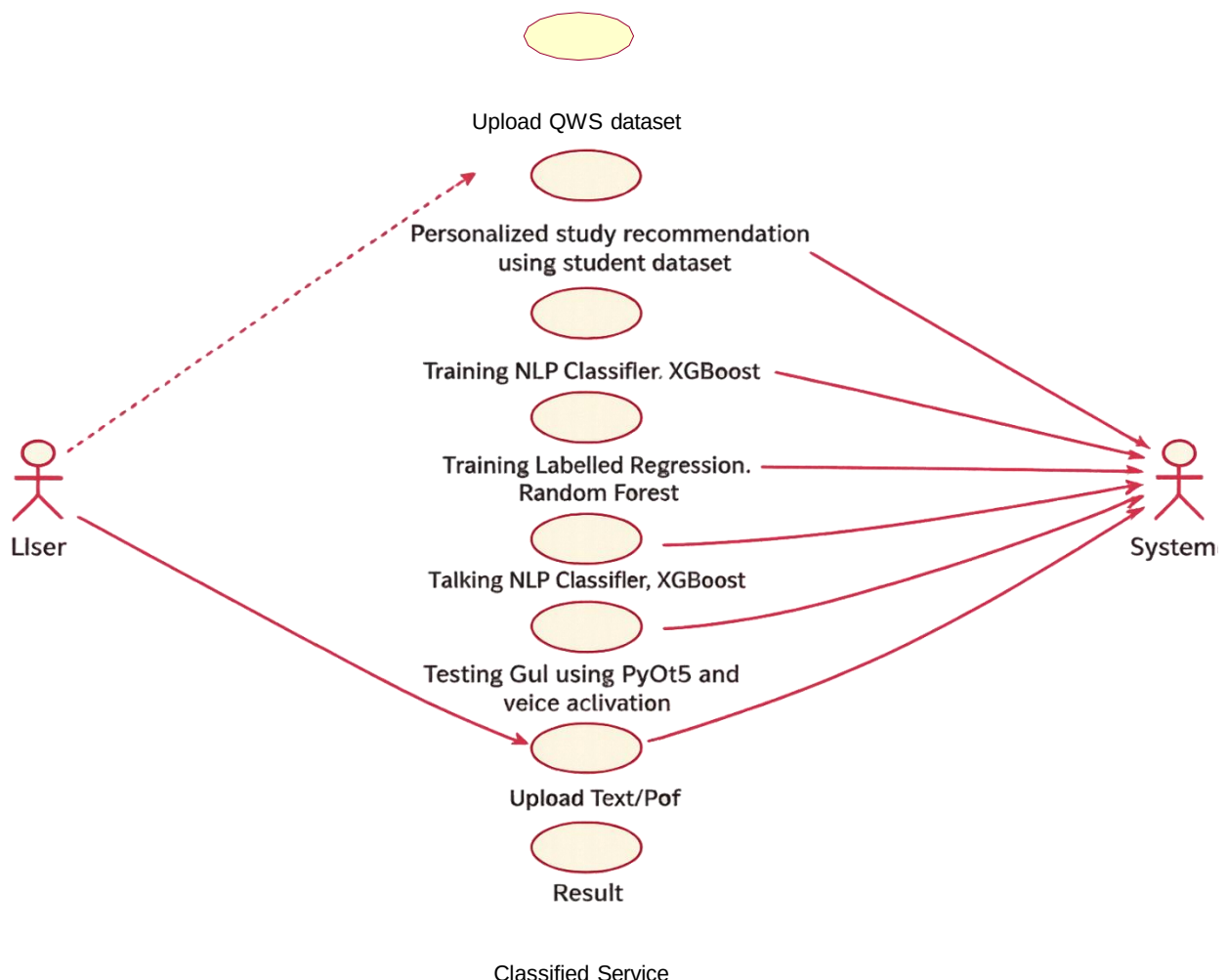


Figure 2 Use case Diagram

10.2 CLASS DIAGRAM

As it belongs to specific types within the Unified Modeling Language (UML), a class diagram is categorized as a static structure diagram in computer software construction. This is a system modeling method that models a particular system by displaying its classes (or the sets of objects), their features, methods (or functions), and their inter-relationships.

Class diagrams portray the containment and interaction of classes and provide no ambiguity about the system's blueprint. This is why they are a valuable resource for the analysis and design of object-oriented software systems.

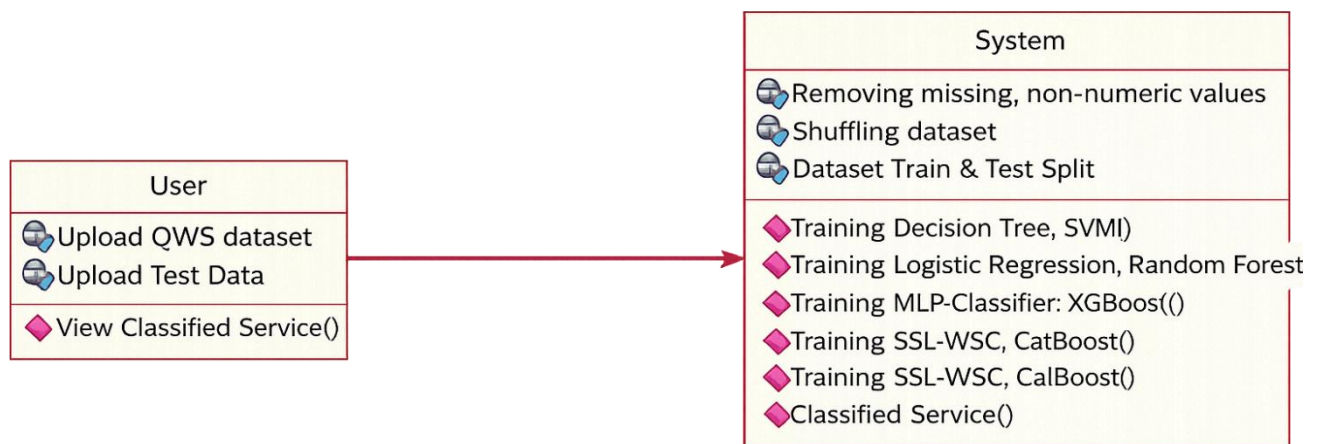


Figure 3 Class Diagram

10.3 SEQUENCE DIAGRAM

UML Sequence diagram belongs to a family of interaction diagrams – dynamic models, and documentation as well – that demonstrate how different processes work, connect and in which order they do so.

Sequence or an event diagram, it is in most cases called an event scenario or timing diagram. It, therefore, provides an elaborated view of the chronological order of events, how objects cooperated in functions over a given period in order to realize a particular process, or in other words, how the objects of the given system behaved during the realized process. These diagrams are great for depicting the behavior of the system and the subtleties of its interactions.

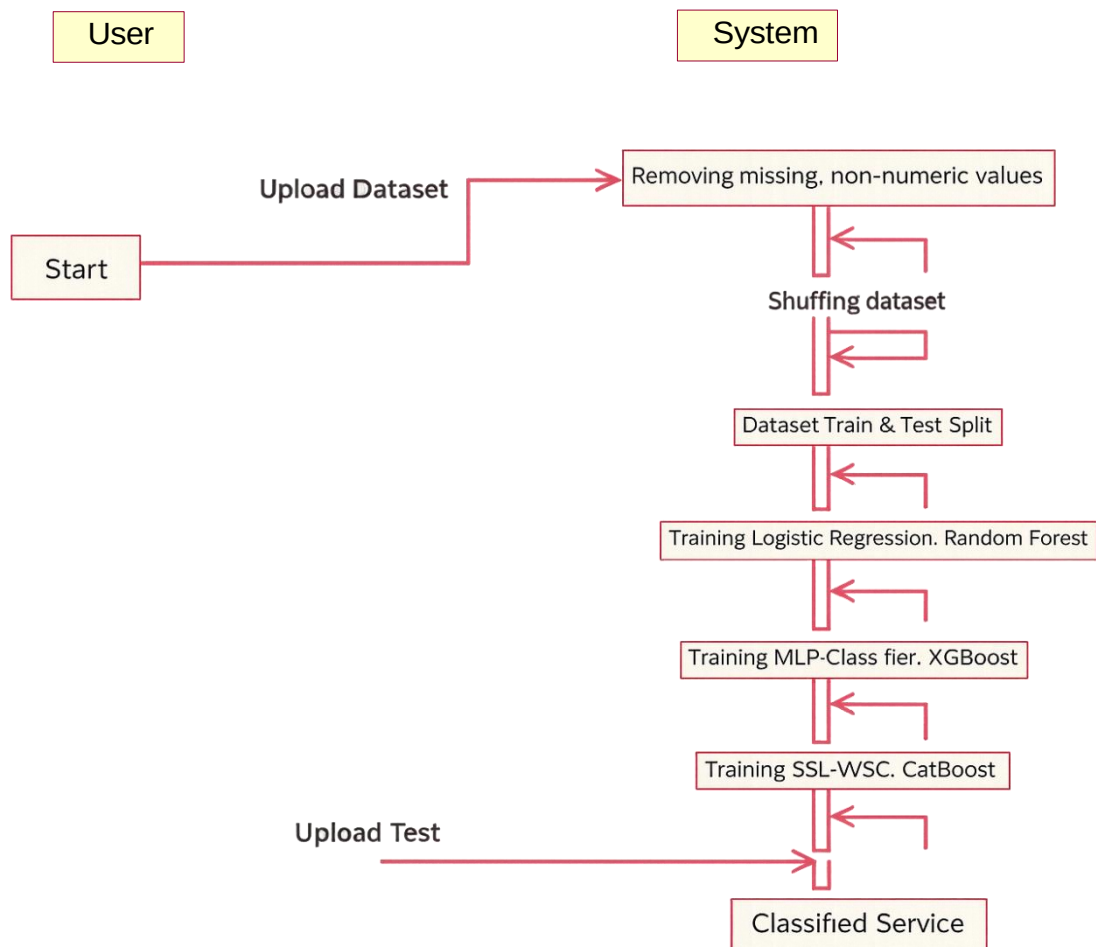


Figure 4 Sequence Diagram

10.4 COLLABORATION DIAGRAM

A collaboration diagram UML is developed to show the links and communication between objects in the system. As opposed to sequence diagrams that chronologically document the sending of messages, collaboration diagrams concentrate on the design and construction of the objects which are the center of the communication at the time of its development.

System objects are endowed with different characteristics, and many of them interrelate with one another and form relationships. The collaboration diagram or the communication diagram depicts these relations graphically in order to give information about the object structure of the system. It assists in appreciating the reason for the existence of some objects and their mode of integration with others in the performance of certain defined functions.

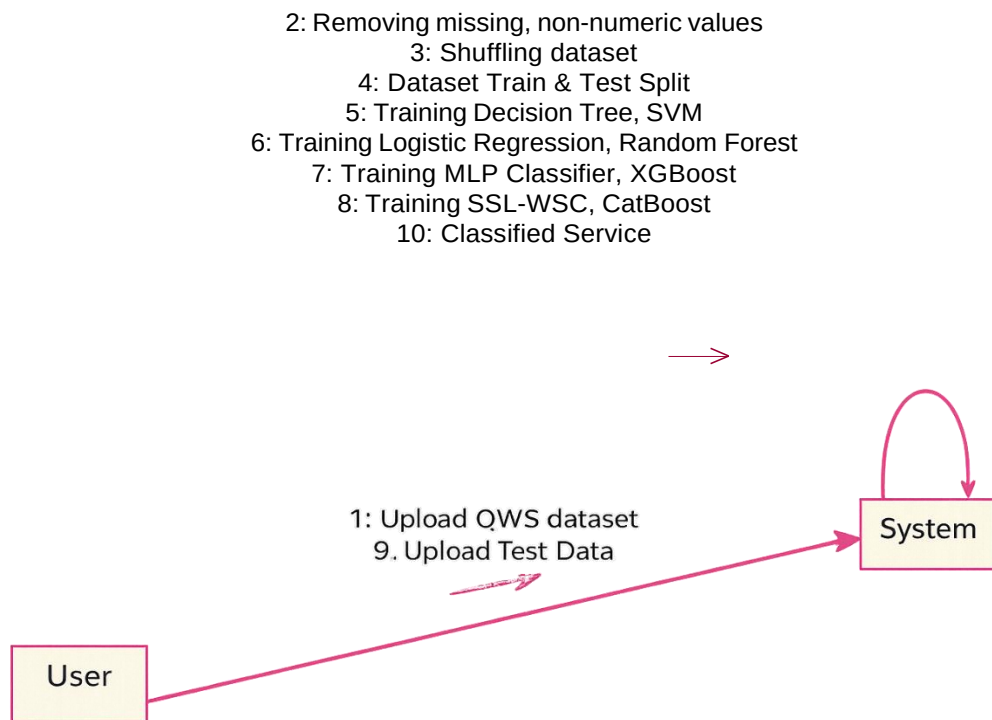


Figure 5 Collaboration Diagram

10.5 ACTIVITY DIAGRAM:

The activity diagram also contains cases of the communication and interactions of objects for the functional purpose. They include a variety of cases such as events and actions in terms of how they flow as well as how they reach the various systems edges or items.

UML activity diagrams outline the business requirements of a process from the user perspective in high level details and also the operational processes of the system explaining how different functional units operate in the context of sequential stages. They are a logically inseparable portion of each process model considering the order of operations and making decisions at each logical point associated with movement of information.

An activity diagram is a fundamental tool in both systems modeling and business processes modeling since it allows any operation to be tracked in a sequential visual representation of its flow.

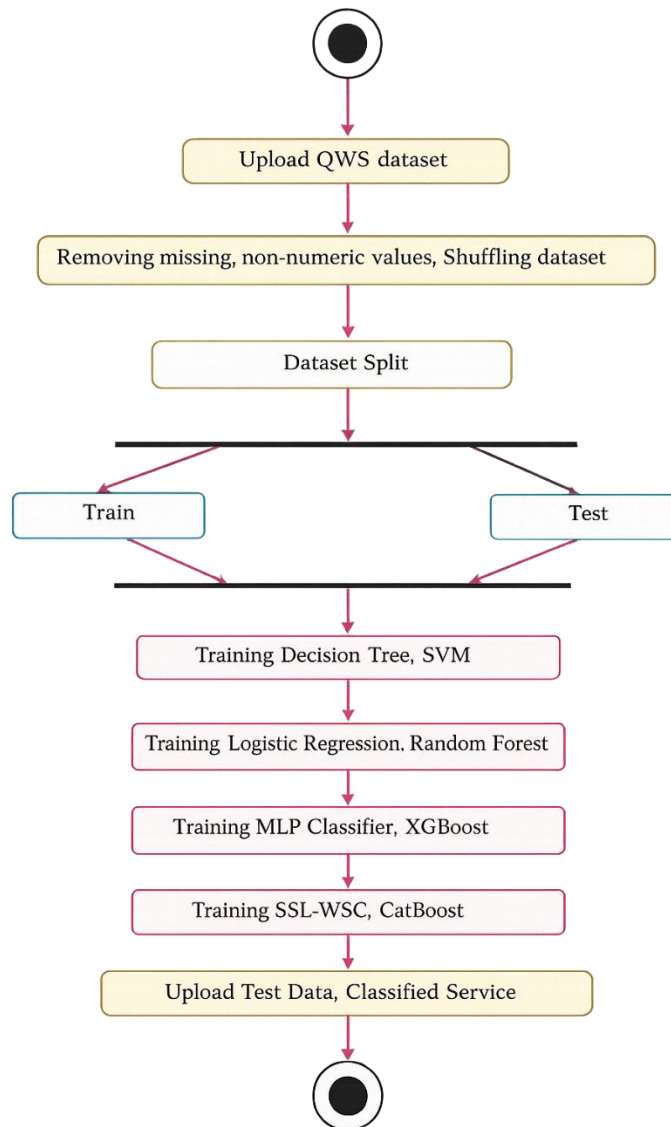


Figure 6 Activity Diagram

11. TECHNOLOGY DESCRIPTIONS

Python Introduction

Python is general purpose high level dynamic interpreted programming language that is used for many applications. Python programming requires a different set of skills since it supports the object oriented programming approach. Python is clear, straightforward, and offers a number of high level data structures making it a preferred language for programmers.

The syntax of the Python language is easy to understand. Its dynamic typing combined with the interpreted nature allows quick jump in application development and scripting. It provides support for multiple paradigms including object oriented, imperative, functional, as well as procedural programming styles to accommodate different project requirements.

Even if Python has not been targeted for a specific area like web programming, it is possible to utilize its multi-dimensional scope in web design, company solutions, implementing 3D CAD, data analysis, and other purposes. One interesting characteristic of the Python programming language is the fact that it is dynamically typed meaning that variables do not need to be explicitly declared. For example, it is possible to do `a = 10` directly.

Python proves beneficial to the user in more than one way - it did away with the need for a compilation step which in turn minimized the amount of time required for debugging and testing. Due to the efficiency of the edit-test-debug cycle, Python makes the ideal language for prototyping and tweaking designs.

11.1 PYTHON FEATURES

Python Characteristics Easy to learn and easy to use:

Being beginner oriented and developer 100 percent focused, python is relatively easy to learn and use.

Expressive Language:

The coding language of Python is declared as highly expressive as it is claimed that it is easy to read and to write.

Interpreted Language:

The reason for this is because Python makes use of an interpreter which takes in the line of code and executes it although it may bring out a few challenges in the debugging of codes.

Cross-platform Compatibility:

This goes to show that the system has the capacity of being used in different kinds of operating systems such as Windows, Linux, Unix and mac operating systems.

Free and Open Source:

Any person who wishes to avail Python can find it free for downloading and It is easy to acquire and modify the source code.

Object-Oriented Programming:

Object-oriented programming is achievable using Python as it provides support for classes and objects among other implementations.

11.2 PYTHON APPLICATIONS

Since Python is a multipurpose language, it can be used in almost all spheres of software development. Other notable fields where Python has its application include:

1. **web Applications:**

With its libraries for internet protocols as HTML, XML, JSON, and many others, and frameworks such as Django, Flask, Pyramid, Python is among the most popular languages for web development. Such include PythonWikiEngines, Pocoo and others.

2. **Software Development:**

Programming in Python is widely used in software development as an auxiliary language for such activities as build control, testing, management, etc.

3. **Scientific and Numeric Computing:**

Libraries like **SciPy**, **Pandas**, and **IPython** are popular for scientific computing, engineering, and mathematical analysis.

4. **Audio/Video Applications:**

Python can develop multimedia apps like TimPlayer or cplay.

11.3 WHY PYTHON

Python is a versatile, user-friendly, and powerful programming language widely used across industries. Here's why Python is so popular:

- **Cross-Platform Compatibility:** Python runs on various platforms, including Windows, Mac, Linux, and Raspberry Pi, making it highly portable.
- **Simple and Readable Syntax:** Python's syntax is simple, clean, and similar to the English language, making it easier for developers to learn and write code.
- **Less Code, More Functionality:** Python allows developers to accomplish tasks with fewer lines of code compared to other programming languages.
- **Interpreter-Based Language:** Python runs on an interpreter, meaning that code can be executed immediately after being written. This makes prototyping faster and more efficient.
- **Multiple Programming Paradigms:** Python supports procedural, object-oriented, and functional programming, offering flexibility to developers.

12. SAMPLE CODE

#import required classes and packages

```
from sklearn.metrics import confusion_matrix

import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler

from scipy.spatial.distance import mahalanobis

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn import svm

from xgboost import XGBClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.neural_network import MLPClassifier

from catboost import CatBoostClassifier

from sklearn.metrics import precision_score

from sklearn.metrics import recall_score

from sklearn.metrics import f1_score

import seaborn as sns

import matplotlib.pyplot as plt

import warnings

warnings.filterwarnings('ignore')

In [121]:

#loading and displaying QWS dataset

dataset = pd.read_csv("Dataset/qws1.txt")

#visualizing class labels count found in full dataset
```

```
labels, count = np.unique(dataset['Class'], return_counts = True)
```

```
labels = ['Platinum', 'Gold', 'Silver', 'Bronze']
```

```
height = count
```

```
bars = labels
```

```
y_pos = np.arange(len(bars))
```

```
plt.figure(figsize = (6, 3))
```

```
plt.bar(y_pos, height)
```

```
plt.xticks(y_pos, bars)
```

```
plt.xlabel("Dataset Class Label Graph")
```

```
plt.ylabel("Count")
```

```
plt.show()
```

#dataset preprocessing techniques like removing missing values, non-numeric values and then shuffling dataset

```
dataset.drop(['Service_Name', 'WSDL_Address'], axis = 1,inplace=True)
```

```
dataset.fillna(0, inplace = True)#replace missing values with 0
```

```
dataset = dataset.values
```

```
X = dataset[:,0:dataset.shape[1]-1]
```

```
Y = dataset[:,dataset.shape[1]-1]
```

```
indices = np.arange(X.shape[0])
```

```
np.random.shuffle(indices)#shuffle dataset to manage all classes in train and test
```

```
X = X[indices]
```

```
Y = Y[indices]
```

```
print("Dataset Processing Completed")
```

Dataset Processing Completed

In [124]:

```
#split dataset into train and test
```

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
```

```
print("Dataset Train & Test Split")  
  
print("80% images used to train algorithms : "+str(X_train.shape[0]))  
  
print("20% image used to train algorithms : "+str(X_test.shape[0]))
```

Dataset Train & Test Split

80% images used to train algorithms : 291

20% image used to train algorithms : 73

In [125]:

#define global variables to save accuracy and other metrics

```
accuracy = []
```

```
precision = []
```

```
recall = []
```

```
fscore = []
```

In [126]:

#function to calculate accuracy and other metrics

```
def calculateMetrics(algorithm, predict, y_test):  
  
    a = accuracy_score(y_test,predict)*100  
  
    p = precision_score(y_test, predict,average='macro') * 100  
  
    r = recall_score(y_test, predict,average='macro') * 100  
  
    f = f1_score(y_test, predict,average='macro') * 100  
  
    accuracy.append(a)  
  
    precision.append(p)  
  
    recall.append(r)  
  
    fscore.append(f)  
  
    print(algorithm+" Accuracy : "+str(a))  
  
    print(algorithm+" Precision : "+str(p))  
  
    print(algorithm+" Recall   : "+str(r))  
  
    print(algorithm+" FScore   : "+str(f))
```

```

conf_matrix = confusion_matrix(y_test, predict)

plt.figure(figsize =(6, 3))

ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True,
cmap="viridis" ,fmt ="g");

ax.set_ylim([0,len(labels)])

plt.title(algorithm+" Confusion matrix")

plt.xticks(rotation=90)

plt.ylabel('True class')

plt.xlabel('Predicted class')

plt.show()

```

In [127]:

#function to train DecisionTreeClassifier algorithm on training data and evaluate performance on test data

```
dt = DecisionTreeClassifier(max_depth=3, max_features=1)
```

```
dt.fit(X_train, y_train)
```

#perform prediction on test data

```
predict = dt.predict(X_test)
```

#call this function to calculate accuracy and other metrics

```
calculateMetrics("Decision Tree", predict, y_test)
```

Decision Tree Accuracy : 38.35616438356164

Decision Tree Precision : 26.16071428571429

Decision Tree Recall : 40.1672149122807

Decision Tree FScore : 28.372627947096028

#function to train SVM algorithm on training data and evaluate performance on test data

```
svm_cls = svm.SVC(probability=True)
```

```
svm_cls.fit(X_train, y_train)
```

#perform prediction on test data

```
predict = svm_cls.predict(X_test)

#call this function to calculate accuracy and other metrics
calculateMetrics("SVM", predict, y_test)

SVM Accuracy : 24.65753424657534

SVM Precision : 20.217917675544793

SVM Recall   : 27.527412280701757

SVM FScore   : 15.86971586971587

#function to train LogisticRegression algorithm on training data and evaluate  
performance on test data

lr = LogisticRegression(max_iter=1500, solver='liblinear', tol=0.2)

lr.fit(X_train, y_train)

#perform prediction on test data

predict = lr.predict(X_test)
```

13. TESTING

13.1 INTRODUCTION

Software engineers draw a distinction between software fault and software failure. A fault will mean an error that is either a bug in programming or a defect in a certain program's semantic. A fault may or may not result in a failure. A failure on the other hand is such that a software has not performed as expected by the end user. Under certain conditions such as when the faulty code is run on the CPU, or even when it is migrated to another hardware machine or when it is enhanced with new capabilities, faults become failures.

Testing a software application is always essential in the process of analyzing the software with the aim of validating its quality and providing the stakeholders with relevant information. It states the errors that it contains and so guarantees that a software has a certain amount of standards that it fulfills with users and system requirements.

13.2 System Testing and Implementation

System testing is concerned with testing the logic of individual module codes and debugging them before their assembly into larger subsystems. This involves:

1. **Module Integration Testing:** This tests the linkages of modules to one another.
2. **System Testing:** This is done to check that the whole system conforms to the requirements provided.
3. **Acceptance Testing:** This goal is to represent the abilities of the system to the customers.

There are two methods that could be used in selecting test cases which include:

- **Black Box Testing** - Testing of the external behavior with respect to the system requirements specification.
- **White Box Testing** - Concentrates on the assessment of internal logic and aims at coverage of the code.

13.3 Testing Techniques

Testing discovers mistakes made in the software by executing the software under conditions that were planned in advance. Two main approaches are:

1. Black Box Testing:

Emphasizes functional specification and assesses external behavior without any interference on internal logic. It detects errors such as interface error, the database access error, performance as well.

2. White Box Testing:

Deals with the internal logic by scrutinizing module flow graphs and analyzing logical paths, loops and data structures to be tested

13.4 Testing Strategies

To ensure their applications are free from errors and comply with the required functionality, the effective performance of software testing is paramount. Given below are the top primary kinds of software testing strategies used in the software development process:

Unit Testing

Unit testing tends to assure the correctness of the implementation of individual units or components of the software. It is usually conducted to ensure the correctness of internal logic, and that for a particular input element expected output is given. The unit testing is said to be structural and invasive since it depends on the knowledge of the structure of the software developed.

The main objective of developers when using unit testing is to ensure the validity of each decision branch and each internal code path. This guarantees all business logic and configurations function as intended. Unit testing is usually done at the level of components and it helps in detecting faults very early in the development cycle.

A good example would be a system with three modules: a Reputation Module, Route Discovery Module and Audit Module. All of these systems would be put through a separate test. Errors are recognized during this stage, patches are made and the patched executable units are checked before they are incorporated.

Integration Testing

Integration testing is usually done after unit testing and it is concerned with putting several units or components together into one cohesive system as they were meant to work. It is event driven, concentrating on functional interconnections and ultimately the interaction of the components of the system.

While unit testing touches on the performance of each individual component, unit testing is aimed at confirming that all the component systems do not create problems when put together. These tests determine whether there are discrepancies in attached data and also the communication lines between modules so as to enhance unity and harmony in system execution.

System Testing

As the name states, system testing tests the final, integrated software product to ensure everything is functioning according to the requirements. It deals with the global layout and just like the end to end testing, guarantees normal results given the particular components are used in a realistic way.

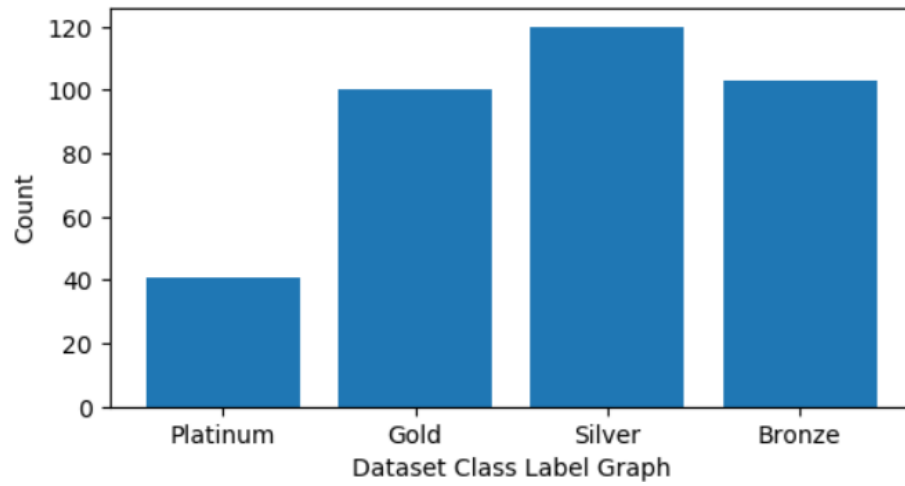
One example of system testing is configuration-oriented system integration testing, which checks how processes interact at various integration points. System testing focuses on process flows and assesses whether the system is reliable and meets the required expectations.

Functional Testing

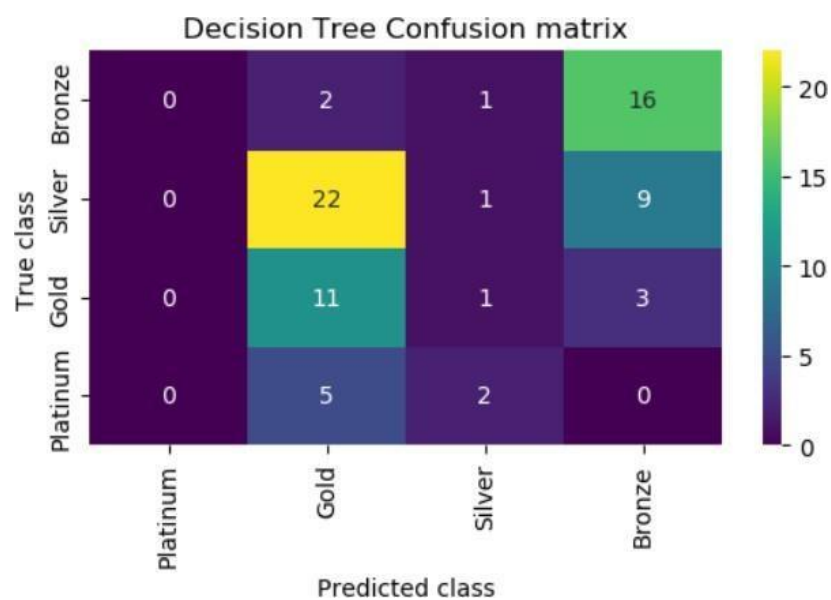
Functional testing establishes whether the system implemented meets the business and technical requirements stated in system documents and users' manuals. It offers a systematic approach for verifying that all required functions have been implemented correctly and as expected by users.

Functional testing was done by using each business process or feature and user interaction against documented requirements. This helps ensure that all features work as they are intended to work without any problems.

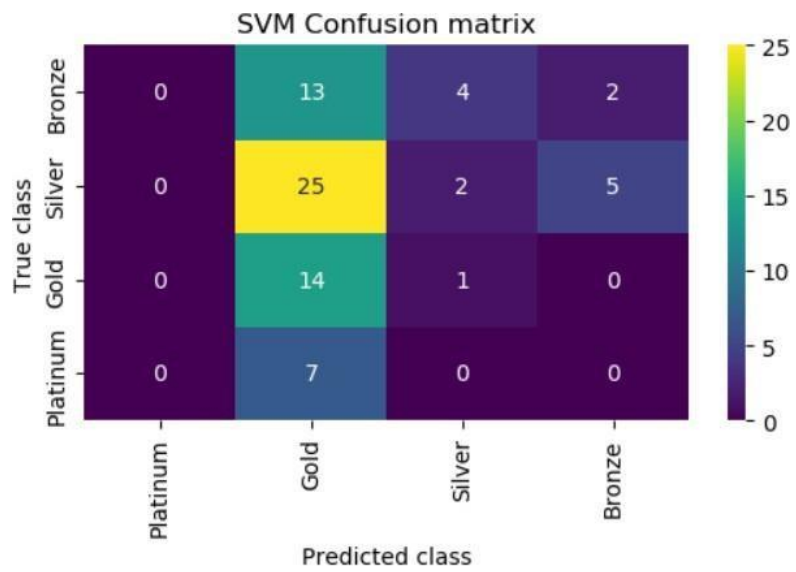
14. SCREENSHOTS



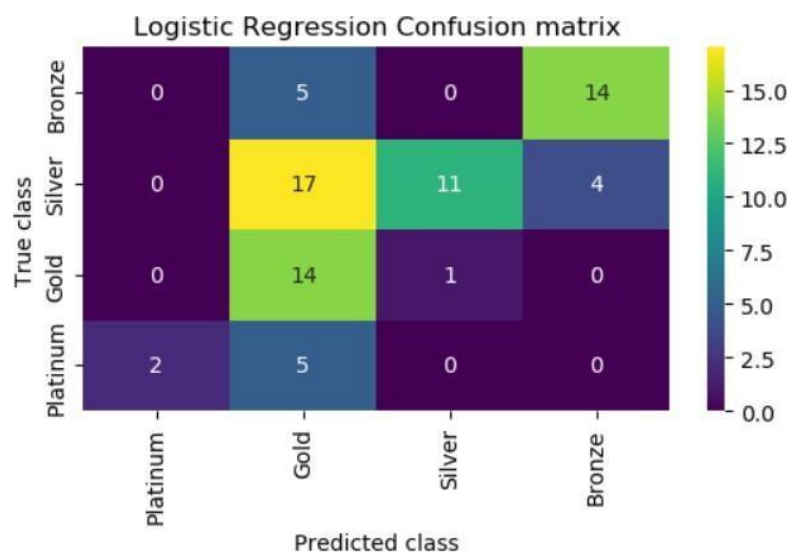
Screen 1 Dataset Class Label Graph



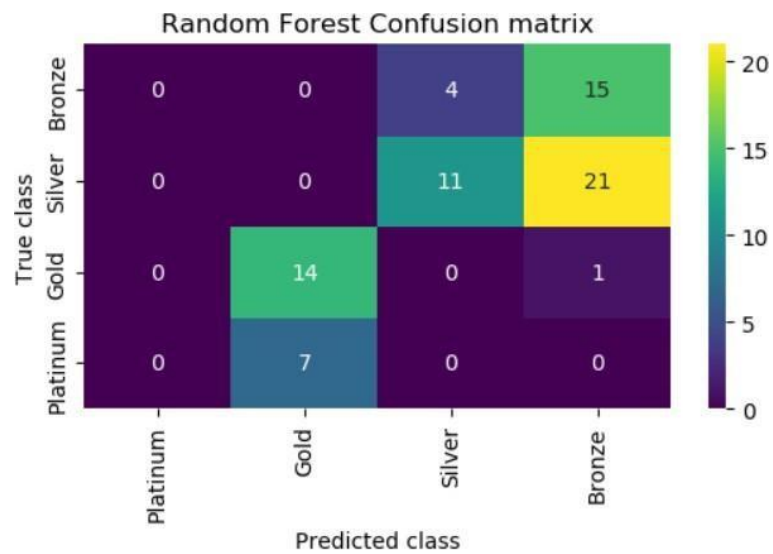
Screen 2 Decision Tree



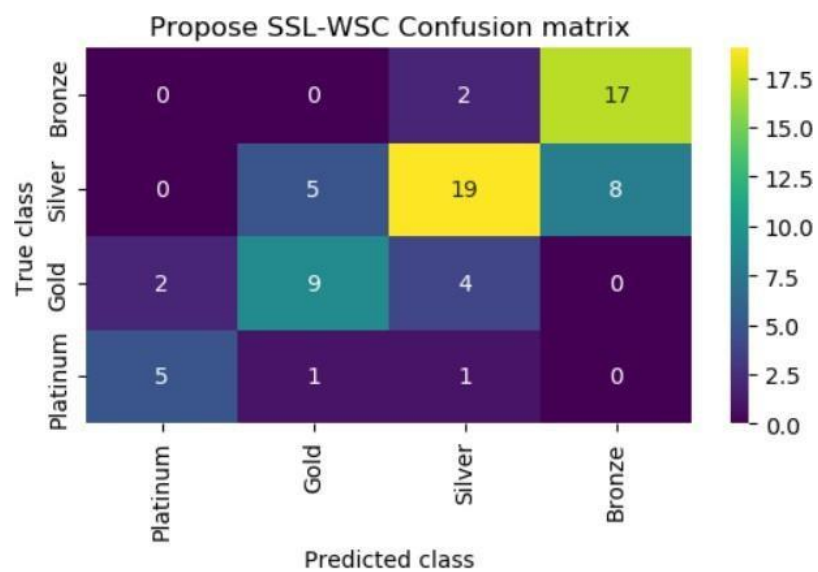
Screen 3 SVM



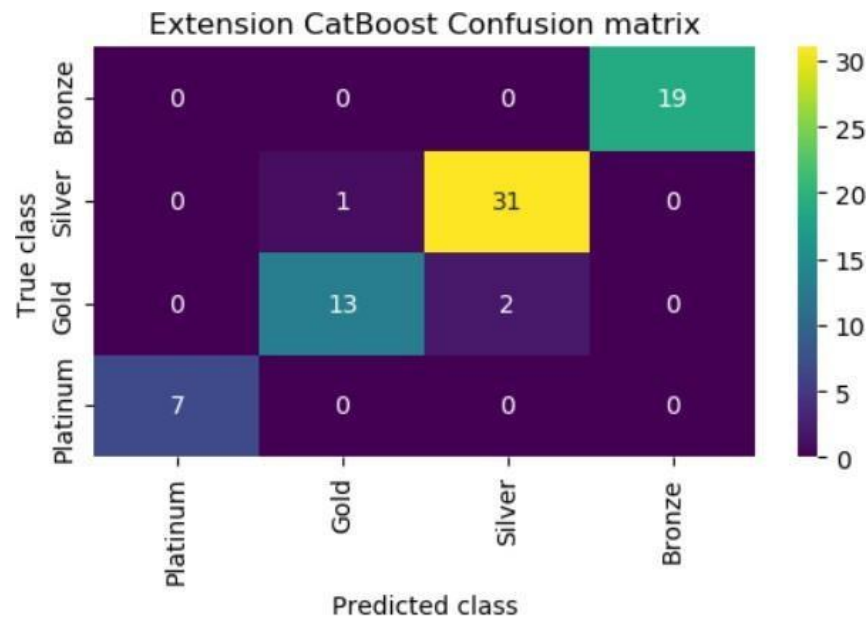
Screen 4 Logistic Regression



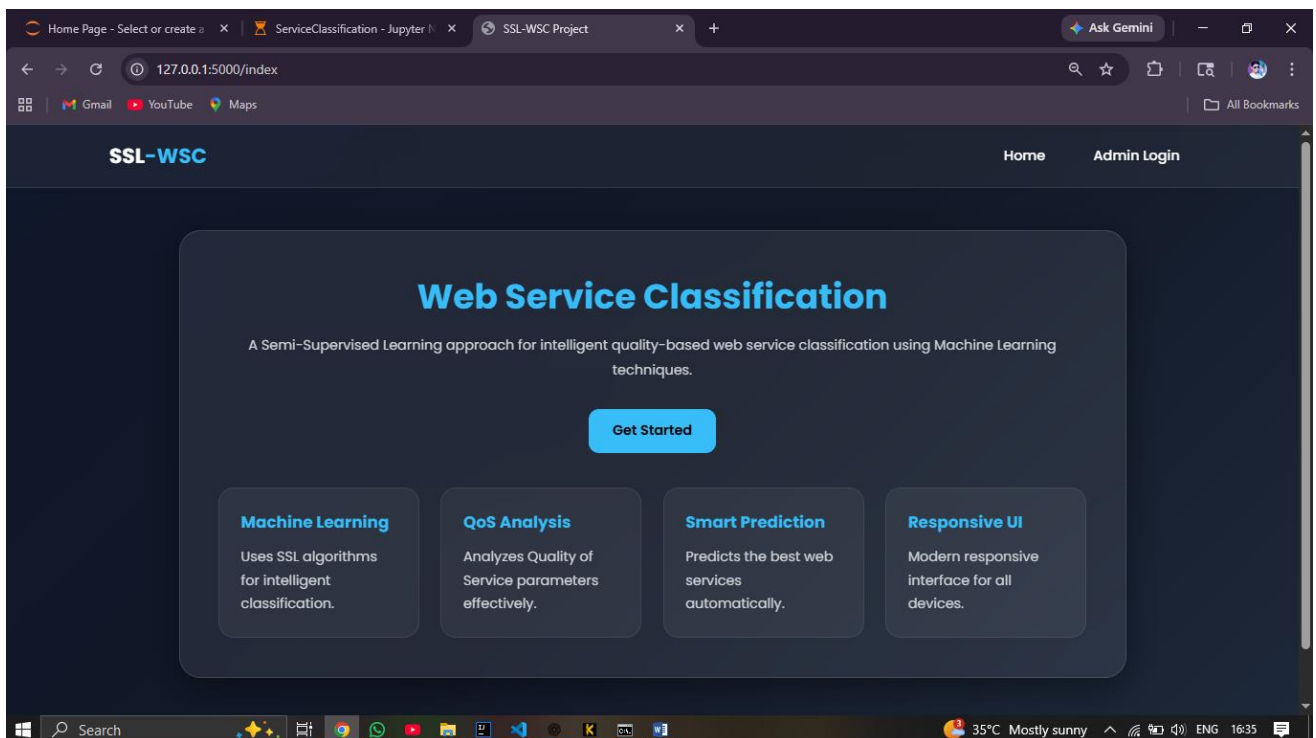
Screen 5 [Random Forest](#)



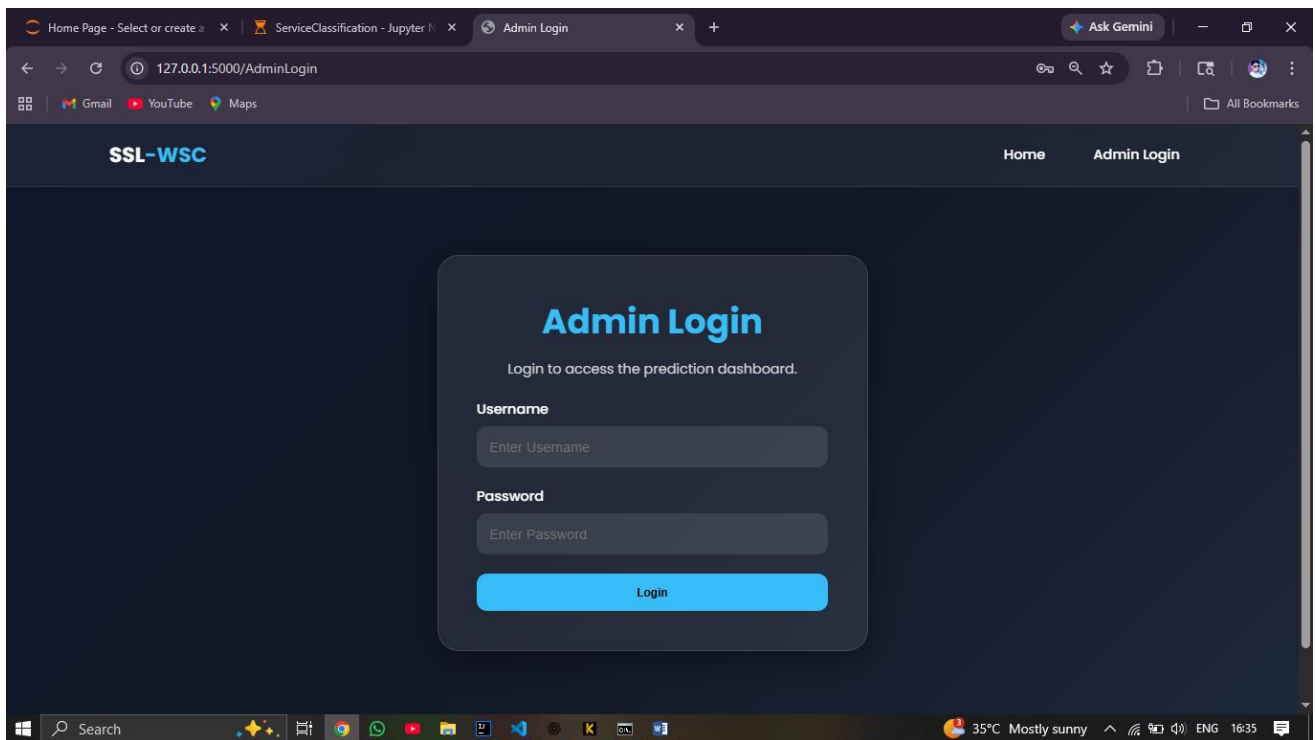
Screen 6 SSL-WSC



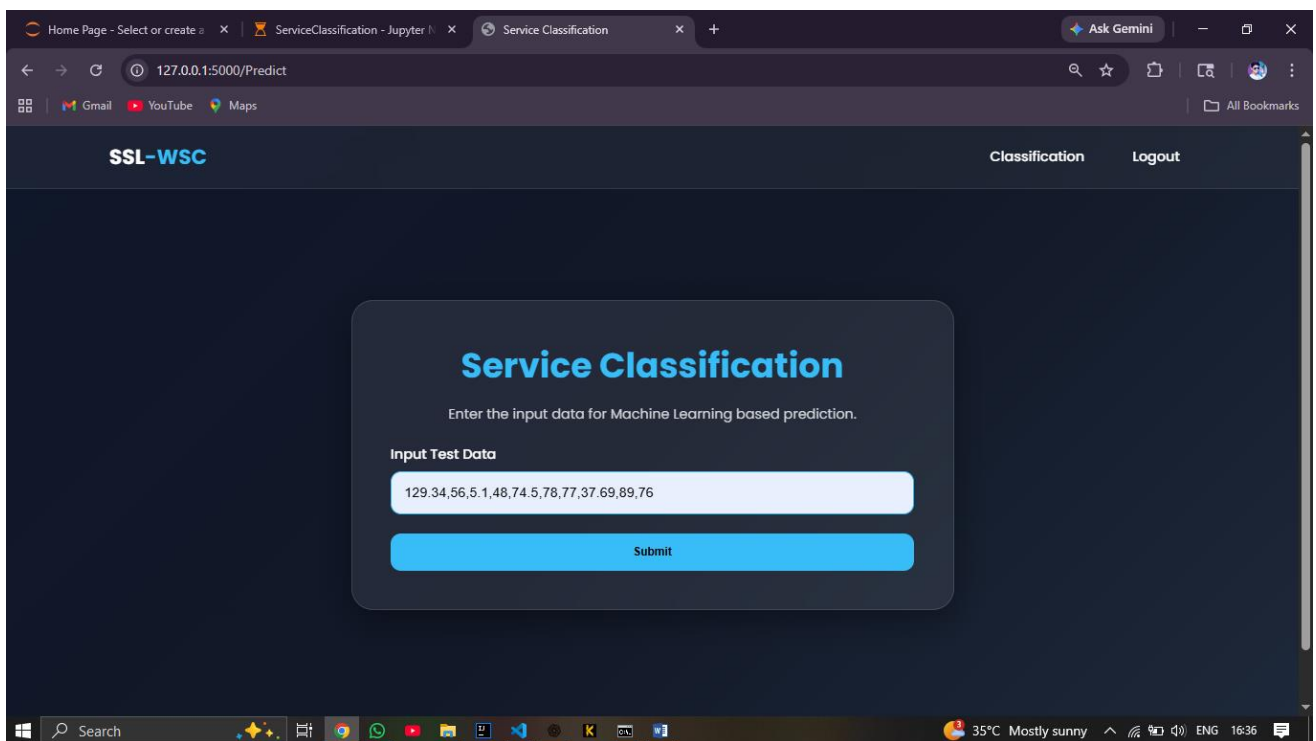
Screen 7 CatBoost



Screen 8 Home Page



Screen 9 Admin Login



Screen 10 Predicted Web service as Silver

15. CONCLUSION

The SSL-WSC algorithm provides a reliable approach to web service classification with minimal labeled data. By integrating advanced distance-based scoring techniques, it significantly improves prediction accuracy while reducing dependency on extensive labeled datasets. The proposed model outperforms traditional classifiers and is further enhanced by CATBOOST, achieving over 90% accuracy. This research effectively balances automation and accuracy, making it a scalable solution for web service ranking. Future improvements could explore deep learning models like CNN and Bi-LSTM for even better classification precision. The proposed approach ensures that users can efficiently select high-quality web services with minimal effort.

.

16. GLOSSARY

Artificial Intelligence (AI):

A field of computer science focused on developing systems that can perform tasks requiring human intelligence, such as decision-making, learning, and problem-solving.

Automatic Speech Recognition (ASR):

A technology that converts spoken language into written text using machine learning models.

CatBoost (Categorical Boosting):

A gradient boosting algorithm designed to handle categorical data efficiently and reduce overfitting.

Computer-Aided Design (CAD):

Software used for designing and modeling objects in engineering and architecture.

Extensible Markup Language (XML):

A markup language used to store and transport data in a structured format.

Generative Adversarial Network (GAN):

A deep learning model consisting of two networks (generator and discriminator) that compete to generate realistic data samples.

Graph Convolutional Network (GCN):

A neural network model designed to process graph-structured data by capturing relationships between connected nodes.

Graphical User Interface (GUI):

A user interface that allows interaction with a system through visual elements like buttons and menus.

HyperText Markup Language (HTML):

The standard language used to create web pages.

Integrated Development Environment (IDE):

A software application that provides tools for coding, debugging, and testing programs.

JavaScript Object Notation (JSON):

A lightweight data-interchange format used for storing and transmitting data.

K-Nearest Neighbors (KNN):

A simple supervised algorithm that classifies data points based on the majority class of their nearest neighbors.

Machine Learning (ML):

A subset of AI that enables systems to learn from data and improve performance without being explicitly programmed.

Multi-Layer Perceptron (MLP):

A type of artificial neural network consisting of multiple layers of neurons used for complex pattern recognition.

Natural Language Processing (NLP):

A branch of AI that enables machines to understand, interpret, and generate human language.

Non-Functional Requirements (NFR):

System requirements that define performance, usability, reliability, and scalability rather than specific functionalities.

Quality of Web Services (QWS):

A dataset containing various quality attributes (such as response time, availability, and throughput) used to evaluate and classify web services.

Semi-Supervised Learning (SSL):

A hybrid learning approach that uses a small amount of labeled data along with a large amount of unlabeled data for training models.

Semi-Supervised Learning for Web Service Classification (SSL-WSC):

A machine learning approach that combines labeled and unlabeled data to classify web services based on quality attributes, improving accuracy with limited labeled data.

Service-Oriented Architecture (SOA):

A design approach where software components provide services to other components over a network.

Simple Object Access Protocol (SOAP):

A protocol used for exchanging structured information in web services using XML.

Support Vector Machine (SVM):

A supervised learning algorithm used for classification and regression tasks by finding the optimal boundary (hyperplane) between data classes.

Unified Modeling Language (UML):

A standardized modeling language used to visualize, design, and document software systems.

Web Services Description Language (WSDL):

An XML-based language used to describe the functionality and location of web services.

XGBoost (Extreme Gradient Boosting):

An optimized machine learning algorithm that uses gradient boosting techniques for high performance and accuracy.

17. BIBLIOGRAPHY

- [1] K. Zhao, J. Liu, Z. Xu, X. Liu, L. Xue, Z. Xie, Y. Zhou, and X. Wang, “Graph4Web: A relation-aware graph attention network for web service classification,” *J. Syst. Softw.*, vol. 190, Aug. 2022, Art. no. 111324.
- [2] M. Masdari, M. N. Bonab, and S. Ozdemir, “Correction to: QoS-driven metaheuristic service composition schemes: A comprehensive overview,” *Artif. Intell. Rev.*, vol. 55, no. 2, p. 1605, Feb. 2022.
- [3] P. B. Pandharbale, S. N. Mohanty, and A. K. Jagadev, “QoS-aware web services recommendations using dynamic clustering algorithms,” *Int. J. Inf. Syst. Model. Design*, vol. 13, no. 6, pp. 1–16, Sep. 2022.
- [4] M. Hasnain, “Machine learning methods for trust-based selection of web services,” *KSII Trans. Internet Inf. Syst.*, vol. 16, no. 1, pp. 38–59, 2022.
- [5] J. M. Duarte and L. Berton, “A review of semi-supervised learning for text classification,” *Artif. Intell. Rev.*, vol. 56, no. 9, pp. 9401–9469, Sep. 2023.